

Algorithm Design and Analysis

HUS - HKII,
2025-2026

Lecturer: Nguyễn Thị Hồng Minh - Đặng Thị Oanh
Trần Bá Tuấn - Trần Anh Minh

Assignment 2

§ Algorithm Analysis §

Section 1: Objectives

- Students are required to master the fundamental concepts of algorithms and algorithm analysis.
- Students understand how to evaluate the computational complexity of algorithms and the relationships expressed using asymptotic notation.
- Students clearly comprehend methods for proving the correctness of algorithms.

Section 2: Practice

(1) Quy cách nộp bài

- Nộp bài bằng file văn bản hoặc ảnh chụp scan thành PDF nếu bài làm viết tay ra giấy (khuyến khích sinh viên thực hiện trình bày bài làm bằng LaTeX).
- Chương trình viết bằng ngôn ngữ Java hoặc Python. Mã nguồn chương trình, file dữ liệu (nếu có) được đặt trong cùng thư mục, kèm theo hướng dẫn cách thực hiện chương trình nếu cần.
- Tất cả các file liên quan tới bài tập để trong thư mục tên Hw2_MaSinhVien_Hovaten, thư mục được nén thành file.zip cùng tên thư mục.
- Khi hết hạn nộp bài, các bài làm sẽ công bố để thực hiện chấm chéo bài tập.
- Sinh viên không nộp bài sẽ nhận điểm 0 bài tập tuần.
- Sinh viên CÓ GIAN LẬN trong nộp bài tập sẽ bị ĐÌNH CHỈ môn học (điểm 0 cho tất cả các điểm thành phần).

(2) In-class Practice

- Present a brief introduction to algorithm complexity, the relationships between algorithm complexity expressed in asymptotic notation, and the different types of asymptotic notation to students.

Example 1. Find the upper bound for:

a) $f(n) = 4n + 5$.

b) $f(n) = n^2 + 2$.

c) $f(n) = n^4 + 100n^2 + 50$.

d) $f(n) = 2n^3 - 2n^2$.

d) $f(n) = 508$.

The common question from Example 2 to Example 8 is:
What is the complexity of the algorithm?

Example 2. Compute $S = \frac{n * (n - 1)}{2}$.

Example 3.

```
1 for i in range(0, n):
2     print('Current Number', i)
```

Example 4.

```
1 for i in range(0, n):
2     print('Current Number', i)
3     break
```

Example 5.

```
1 def function(n):
2     i = 1
3     while i <= n:
4         i = i * 2
5         print(i)
6 function(100)
```

Example 6.

```
1 for i in range(0, n):
2     for j in range(0, n):
3         print("Value of i, j", i, j)
```

Example 7.

```
1 public void function(int n){
2     int i, j, k, count = 0;
3     for(i = n/2; i <= n; i++)
4         for(j = 1; j + n/2 <= n; j++)
5             for(k = 1; k <= n; k = k * 2)
6                 count++;
7 }
```

Example 8.

```

1 public void function(int n) {
2     int i, j, k, count = 0;
3     for(i = n/2; i <= n; i++)
4         for(j = 1; j <= n; j = 2 * j)
5             for(k = 1; k <= n; k = k * 2)
6                 count++;
7 }

```

Example 9. What is the complexity of the algorithm?

- a) $T(n) = n \log n + 3n + 2$
- b) $T(n) = n \log(n!) + 5n^2 + 6$
- c) $T(n) = 100n + 10n^2$
- d) $T(n) = 50n \log n + n^3 + 100n$
- e) $T(n) = 10n \log n + n(\log n)^2$

Example 10. What is the complexity of the algorithms in the code segments below?

a)

```

1 // Returns the sum of the integers in given array.
2 public static int example1(int[] arr) {
3     int n = arr.length, total = 0;
4     for (int j=0; j < n; j++) // loop from 0 to n-1
5         total += arr[j];
6     return total;
7 }

```

b)

```

1 // Returns the sum of the integers with even index in given
  ↪ array.
2 public static int example2(int[] arr) {
3     int n = arr.length, total = 0;
4     for (int j=0; j < n; j += 2) // note the increment of 2
5         total += arr[j];
6     return total;
7 }

```

c)

```

1 // Returns the sum of the prefix sums of given array.
2 public static int example3(int[] arr) {
3     int n = arr.length, total = 0;
4     for (int j=0; j < n; j++) // loop from 0 to n-1
5         for (int k=0; k <= j; k++) // loop from 0 to j
6             total += arr[j];
7     return total;
8 }

```

d)

```

1 // Returns the sum of the prefix sums of given array.
2 public static int example4(int[] arr) {
3     int n = arr.length, prefix = 0, total = 0;
4     for (int j=0; j < n; j++) { // loop from 0 to n-1
5         prefix += arr[j];
6         total += prefix;
7     }
8     return total;
9 }

```

e)

```

1 // Returns the number of times second array stores sum of
  ↪ prefix sums from first.
2 public static int example5(int[] first, int[] second) {
3     // assume equal-length arrays
4     int n = first.length, count = 0;
5     for (int i=0; i < n; i++) { // loop from 0 to n-1
6         int total = 0;
7         for (int j=0; j < n; j++) // loop from 0 to n-1
8             for (int k=0; k <= j; k++) // loop from 0 to j
9                 total += first[k];
10        if (second[i] == total) count++;
11    }
12    return count;
13 }

```

Exercise 1. Determine the input size and the order of growth of the algorithmic complexity.

Exercise 2.1, Anany's book: Select at least six of the problems to complete.

Exercise 2. Determine the relationships among algorithm complexities using asymptotic notation.

Exercise 2.2, Anany's book: Select at least six of the problems to complete.

Exercise 3. Design the algorithm, prove its correctness, and determine its complexity.

- **Problem:** Construct the **selection sort** algorithm based on the following idea: sort n numbers stored in array a by finding the smallest element of a and swapping it with $a[1]$. Then find the second smallest element of a and swap it with $a[2]$. Continue this process for the remaining $n - 1$ elements of a .
- Write the pseudocode for this algorithm.
- Why is it sufficient to perform the process for only the first $n - 1$ elements instead of all n elements?
- Provide an evaluation of the algorithm's running time in the best case and the worst case using Big-O notation.

- *Implement the program and test it on a sequence with a large number of randomly generated elements. Measure the execution time and plot the relationship between running time and input size n .*
- *(*) Use loop invariants to prove that the algorithm is correct (state the loop invariant and prove that it satisfies the three properties: initialization, maintenance, and termination).*
- *(**) Students are required to propose at least two other **sorting algorithms** and perform the same set of tasks as above.*

For further reading on loop invariants, see Rodney R. Howell's book, "Algorithm: A Top-Down Approach", Chapter 2, to complete the advanced exercise ().*